

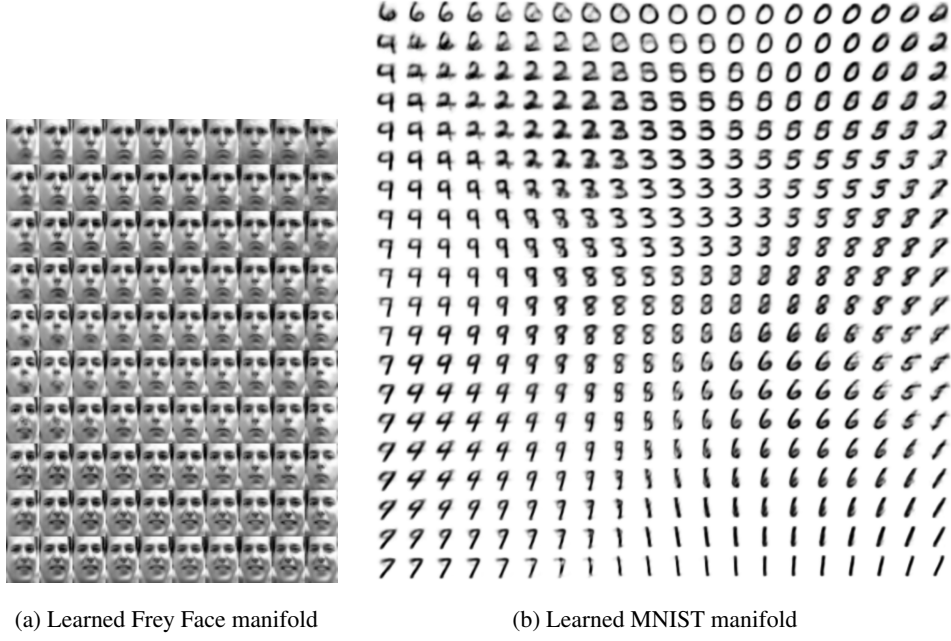


Figure 4: Visualisations of learned data manifold for generative models with two-dimensional latent space, learned with AEVB. Since the prior of the latent space is Gaussian, linearly spaced coordinates on the unit square were transformed through the inverse CDF of the Gaussian to produce values of the latent variables $\mathbf{z}$. For each of these values $\mathbf{z}$, we plotted the corresponding generative $p_{\theta}(\mathbf{x}|\mathbf{z})$ with the learned parameters θ .

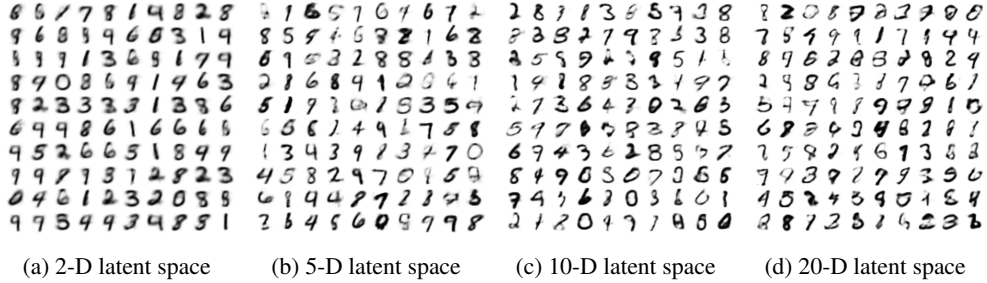


Figure 5: Random samples from learned generative models of MNIST for different dimensionalities of latent space.

B Solution of $-D_{KL}(q_{\phi}(\mathbf{z})||p_{\theta}(\mathbf{z}))$, Gaussian case

The variational lower bound (the objective to be maximized) contains a KL term that can often be integrated analytically. Here we give the solution when both the prior $p_{\theta}(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ and the posterior approximation $q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)})$ are Gaussian. Let J be the dimensionality of $\mathbf{z}$. Let $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ denote the variational mean and s.d. evaluated at datapoint i , and let μ_j and σ_j simply denote the j -th element of these vectors. Then:

$$\begin{aligned}
 \int q_{\theta}(\mathbf{z}) \log p(\mathbf{z}) d\mathbf{z} &= \int \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}, \boldsymbol{\sigma}^2) \log \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I}) d\mathbf{z} \\
 &= -\frac{J}{2} \log(2\pi) - \frac{1}{2} \sum_{j=1}^J (\mu_j^2 + \sigma_j^2)
 \end{aligned}$$

And:

$$\begin{aligned}\int q_{\theta}(\mathbf{z}) \log q_{\theta}(\mathbf{z}) d\mathbf{z} &= \int \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}, \boldsymbol{\sigma}^2) \log \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}, \boldsymbol{\sigma}^2) d\mathbf{z} \\ &= -\frac{J}{2} \log(2\pi) - \frac{1}{2} \sum_{j=1}^J (1 + \log \sigma_j^2)\end{aligned}$$

Therefore:

$$\begin{aligned}-D_{KL}(q_{\phi}(\mathbf{z})||p_{\theta}(\mathbf{z})) &= \int q_{\theta}(\mathbf{z}) (\log p_{\theta}(\mathbf{z}) - \log q_{\theta}(\mathbf{z})) d\mathbf{z} \\ &= \frac{1}{2} \sum_{j=1}^J (1 + \log((\sigma_j)^2) - (\mu_j)^2 - (\sigma_j)^2)\end{aligned}$$

When using a recognition model $q_{\phi}(\mathbf{z}|\mathbf{x})$ then $\boldsymbol{\mu}$ and s.d. $\boldsymbol{\sigma}$ are simply functions of $\mathbf{x}$ and the variational parameters ϕ , as exemplified in the text.

C MLP's as probabilistic encoders and decoders

In variational auto-encoders, neural networks are used as probabilistic encoders and decoders. There are many possible choices of encoders and decoders, depending on the type of data and model. In our example we used relatively simple neural networks, namely multi-layered perceptrons (MLPs). For the encoder we used a MLP with Gaussian output, while for the decoder we used MLPs with either Gaussian or Bernoulli outputs, depending on the type of data.

C.1 Bernoulli MLP as decoder

In this case let $p_{\theta}(\mathbf{x}|\mathbf{z})$ be a multivariate Bernoulli whose probabilities are computed from $\mathbf{z}$ with a fully-connected neural network with a single hidden layer:

$$\begin{aligned}\log p(\mathbf{x}|\mathbf{z}) &= \sum_{i=1}^D x_i \log y_i + (1 - x_i) \cdot \log(1 - y_i) \\ \text{where } \mathbf{y} &= f_{\sigma}(\mathbf{W}_2 \tanh(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2)\end{aligned}\tag{11}$$

where $f_{\sigma}(\cdot)$ is the elementwise sigmoid activation function, and where $\boldsymbol{\theta} = \{\mathbf{W}_1, \mathbf{W}_2, \mathbf{b}_1, \mathbf{b}_2\}$ are the weights and biases of the MLP.

C.2 Gaussian MLP as encoder or decoder

In this case let encoder or decoder be a multivariate Gaussian with a diagonal covariance structure:

$$\begin{aligned}\log p(\mathbf{x}|\mathbf{z}) &= \log \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\sigma}^2 \mathbf{I}) \\ \text{where } \boldsymbol{\mu} &= \mathbf{W}_4 \mathbf{h} + \mathbf{b}_4 \\ \log \boldsymbol{\sigma}^2 &= \mathbf{W}_5 \mathbf{h} + \mathbf{b}_5 \\ \mathbf{h} &= \tanh(\mathbf{W}_3 \mathbf{z} + \mathbf{b}_3)\end{aligned}\tag{12}$$

where $\{\mathbf{W}_3, \mathbf{W}_4, \mathbf{W}_5, \mathbf{b}_3, \mathbf{b}_4, \mathbf{b}_5\}$ are the weights and biases of the MLP and part of $\boldsymbol{\theta}$ when used as decoder. Note that when this network is used as an encoder $q_{\phi}(\mathbf{z}|\mathbf{x})$, then $\mathbf{z}$ and $\mathbf{x}$ are swapped, and the weights and biases are variational parameters ϕ .

D Marginal likelihood estimator

We derived the following marginal likelihood estimator that produces good estimates of the marginal likelihood as long as the dimensionality of the sampled space is low (less than 5 dimensions), and sufficient samples are taken. Let $p_{\theta}(\mathbf{x}, \mathbf{z}) = p_{\theta}(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})$ be the generative model we are sampling from, and for a given datapoint $\mathbf{x}^{(i)}$ we would like to estimate the marginal likelihood $p_{\theta}(\mathbf{x}^{(i)})$.

The estimation process consists of three stages:

1. Sample L values $\{\mathbf{z}^{(l)}\}$ from the posterior using gradient-based MCMC, e.g. Hybrid Monte Carlo, using $\nabla_{\mathbf{z}} \log p_{\theta}(\mathbf{z}|\mathbf{x}) = \nabla_{\mathbf{z}} \log p_{\theta}(\mathbf{z}) + \nabla_{\mathbf{z}} \log p_{\theta}(\mathbf{x}|\mathbf{z})$.
2. Fit a density estimator $q(\mathbf{z})$ to these samples $\{\mathbf{z}^{(l)}\}$.
3. Again, sample L new values from the posterior. Plug these samples, as well as the fitted $q(\mathbf{z})$, into the following estimator:

$$p_{\theta}(\mathbf{x}^{(i)}) \simeq \left(\frac{1}{L} \sum_{l=1}^L \frac{q(\mathbf{z}^{(l)})}{p_{\theta}(\mathbf{z}) p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z}^{(l)})} \right)^{-1} \quad \text{where} \quad \mathbf{z}^{(l)} \sim p_{\theta}(\mathbf{z}|\mathbf{x}^{(i)})$$

Derivation of the estimator:

$$\begin{aligned} \frac{1}{p_{\theta}(\mathbf{x}^{(i)})} &= \frac{\int q(\mathbf{z}) d\mathbf{z}}{p_{\theta}(\mathbf{x}^{(i)})} = \frac{\int q(\mathbf{z}) \frac{p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z})}{p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z})} d\mathbf{z}}{p_{\theta}(\mathbf{x}^{(i)})} \\ &= \int \frac{p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z})}{p_{\theta}(\mathbf{x}^{(i)})} \frac{q(\mathbf{z})}{p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z})} d\mathbf{z} \\ &= \int p_{\theta}(\mathbf{z}|\mathbf{x}^{(i)}) \frac{q(\mathbf{z})}{p_{\theta}(\mathbf{x}^{(i)}, \mathbf{z})} d\mathbf{z} \\ &\simeq \frac{1}{L} \sum_{l=1}^L \frac{q(\mathbf{z}^{(l)})}{p_{\theta}(\mathbf{z}) p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z}^{(l)})} \quad \text{where} \quad \mathbf{z}^{(l)} \sim p_{\theta}(\mathbf{z}|\mathbf{x}^{(i)}) \end{aligned}$$

E Monte Carlo EM

The Monte Carlo EM algorithm does not employ an encoder, instead it samples from the posterior of the latent variables using gradients of the posterior computed with $\nabla_{\mathbf{z}} \log p_{\theta}(\mathbf{z}|\mathbf{x}) = \nabla_{\mathbf{z}} \log p_{\theta}(\mathbf{z}) + \nabla_{\mathbf{z}} \log p_{\theta}(\mathbf{x}|\mathbf{z})$. The Monte Carlo EM procedure consists of 10 HMC leapfrog steps with an automatically tuned stepsize such that the acceptance rate was 90%, followed by 5 weight updates steps using the acquired sample. For all algorithms the parameters were updated using the Adagrad stepsizes (with accompanying annealing schedule).

The marginal likelihood was estimated with the first 1000 datapoints from the train and test sets, for each datapoint sampling 50 values from the posterior of the latent variables using Hybrid Monte Carlo with 4 leapfrog steps.

F Full VB

As written in the paper, it is possible to perform variational inference on both the parameters θ and the latent variables $\mathbf{z}$, as opposed to just the latent variables as we did in the paper. Here, we'll derive our estimator for that case.

Let $p_{\alpha}(\theta)$ be some hyperprior for the parameters introduced above, parameterized by α . The marginal likelihood can be written as:

$$\log p_{\alpha}(\mathbf{X}) = D_{KL}(q_{\phi}(\theta) || p_{\alpha}(\theta|\mathbf{X})) + \mathcal{L}(\phi; \mathbf{X}) \quad (13)$$

where the first RHS term denotes a KL divergence of the approximate from the true posterior, and where $\mathcal{L}(\phi; \mathbf{X})$ denotes the variational lower bound to the marginal likelihood:

$$\mathcal{L}(\phi; \mathbf{X}) = \int q_{\phi}(\theta) (\log p_{\theta}(\mathbf{X}) + \log p_{\alpha}(\theta) - \log q_{\phi}(\theta)) d\theta \quad (14)$$

Note that this is a lower bound since the KL divergence is non-negative; the bound equals the true marginal when the approximate and true posteriors match exactly. The term $\log p_{\theta}(\mathbf{X})$ is composed of a sum over the marginal likelihoods of individual datapoints $\log p_{\theta}(\mathbf{X}) = \sum_{i=1}^N \log p_{\theta}(\mathbf{x}^{(i)})$, which can each be rewritten as:

$$\log p_{\theta}(\mathbf{x}^{(i)}) = D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) || p_{\theta}(\mathbf{z}|\mathbf{x}^{(i)})) + \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) \quad (15)$$